

Strings

Trie

Estructura

```
#include "../setup/template.h"

// snippet: structure
const int K = 26;

struct Vertex {
    int next[K];
    bool output = false;

    Vertex() {
        fill(begin(next), end(next), -1);
    }
};

vector<Vertex> trie(1);
// snippet: end

// snippet: add_string
void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (trie[v].next[c] == -1) {
            trie[v].next[c] = trie.size();
            trie.emplace_back();
        }
        v = trie[v].next[c];
    }
    trie[v].output = true;
}
// snippet: end
```

Agregar una cadena

```
#include "../setup/template.h"

// snippet: structure
const int K = 26;

struct Vertex {
    int next[K];
    bool output = false;

    Vertex() {
        fill(begin(next), end(next), -1);
    }
};

vector<Vertex> trie(1);
// snippet: end
```

```
// snippet: add_string
void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (trie[v].next[c] == -1) {
            trie[v].next[c] = trie.size();
            trie.emplace_back();
        }
        v = trie[v].next[c];
    }
    trie[v].output = true;
}
// snippet: end
```

String hashing

hash-doble

```
#include "../setup/template.h"
// snippet: hash
struct Hash { //doble hashing
    vector<ll> h1, h2;
    vector<ll> p1, p2;
    ll p=31; //tomamos esta p para evitar colisiones
    Hash(string &s) {
        int n = s.size();
        h1.resize(n+1, 0);
        h2.resize(n+1, 0);
        p1.resize(n+1, 1);
        p2.resize(n+1, 1);
    }
    //hacemos doble hasheo para evitar colisiones
    for (int i = 0; i < n; i++) {
        p1[i+1] = (p1[i] * p) % MOD; //a uno le aplicamos modulo
        p2[i+1] = (p2[i] * p); //el otro ocupamos el modulo implicito de ll
    }
    //uno con mod 10e9+7 y otro con 2e64 que es LLONG_MAX
    int val = s[i] - 'a' + 1; //hasheamos con el ascii

    h1[i+1] = (h1[i] * p + val) % MOD;
    h2[i+1] = (h2[i] * p + val);
}

pair<ll,ll> get(ll l, ll r) { //comparamos los 2 hashing
    ll x1 = (h1[r+1] - h1[l] * p1[r-l+1] % MOD + MOD) % MOD;
    ll x2 = (h2[r+1] - h2[l] * p2[r-l+1]);
    return {x1, x2};
}

};
Hash hs(s); //inicializar
hs.get(i,j); //lugar de sonde se compara
if(hs.get(l,m)==hs.get(i,j)) //si 2 cadenas son iguales
// snippet: end
```

KMP — Knuth-Morris-Pratt

KMP — Knuth-Morris-Pratt

TIME

$O(n + m)$

SPACE

$O(m)$

Cuenta ocurrencias de pat en txt. buildPhi construye el arreglo de fallos: phi[j] = prefijo propio más largo de pat[0..j] que es sufijo. Al fallar retrocede a phi[i-1] en lugar de reiniciar.

```
// KMP — Knuth-Morris-Pratt
// Encuentra todas las ocurrencias de pat en txt
// Complejidad:  $O(n + m)$ 

// snippet: kmp
vll buildPhi(const vll& pat) {
    ll m = pat.size();
    vll phi(m, 0);
    for (ll j = 1, i = 0; j < m; j++) {
        while (i > 0 && pat[i] != pat[j]) i = phi[i - 1];
        if (pat[i] == pat[j]) i++;
        phi[j] = i;
    }
    return phi;
}

ll KMP(const vll& pat, const vll& txt) {
    ll m = pat.size(), n = txt.size();
    if (m == 0) return 0;
    vll phi = buildPhi(pat);
    ll matches = 0;
    for (ll i = 0, j = 0; j < n; j++) {
        while (i > 0 && pat[i] != txt[j]) i = phi[i - 1];
        if (pat[i] == txt[j]) i++;
        if (i == m) { matches++; i = phi[i - 1]; }
    }
    return matches;
}

// snippet: end
```